Industrial Surface Defect Classification

using Transfer Learning (MobileNetV2) on the NEU Surface Defect Dataset

---

Assignment report

Submitted by: M Bulah

Notebook: Transfer_Learning_Surface_Defect_MBulah.ipynb

Framework: Python, PyTorch and torchvision

Date: 28 October 2026

Github link: https://github.com/bulah16/C7-Project

Headline results

Measure	Result
Validation accuracy (best checkpoint, epoch 11)	100.00% (360 / 360)
Validation accuracy (final epoch 12, no selection)	99.72% (359 / 360)
Macro F1-score (best checkpoint)	1.000
Train - validation accuracy gap (final epoch)	+0.21 percentage points
Trainable parameters	329,478 of 2,553,350 (12.9%)
Training time (2-core CPU, 12 epochs)	about 11.5 minutes

1. Executive summary

The goal was to classify steel surface images into six defect types (crazing, inclusion, patches, pitted surface, rolled-in scale and scratches) using transfer learning. A MobileNetV2 network pre-trained on ImageNet was reused as a frozen feature extractor, and only a small new classification head was trained.

The approach worked very well. The best checkpoint classified all 360 validation images correctly, and the model from the last epoch, chosen without any selection, still reached 99.72%. Training accuracy (99.9%) and validation accuracy were almost identical, which points to a model with low bias and low variance: it fits the data and generalises.

The only remaining difficulty is separating inclusions from scratches, two defects that both appear as thin streaks running along the rolling direction. Nearly all low-confidence predictions involve this pair. Because confidence tracks difficulty so well, a simple rule (send anything under 90% confidence to a human inspector) would route only about 1-2% of parts to manual review.

2. Problem and business context

Surface defects introduced during steel manufacturing affect product quality, safety, customer satisfaction, production cost and rework rates. Manual visual inspection is slow, inconsistent between inspectors, and hard to scale.

Problem statement: given a steel surface image, predict the type of surface defect present, using a pre-trained convolutional neural network.

Transfer learning makes this practical because it:

- reuses visual knowledge learned from millions of images instead of training from scratch;
- needs far less labelled data and compute;
- produces a lightweight model suitable for real-time or edge deployment on the production line.

3. Dataset

The NEU Surface Defect Dataset contains grayscale 200 x 200 images of hot-rolled steel strip, captured under industrial conditions. The provided split was used unchanged: 240 training and 60 validation images per class.

Class	Train	Validation	Total
Crazing	240	60	300
Inclusion	240	60	300
Patches	240	60	300
Pitted surface	240	60	300
Rolled-in scale	240	60	300
Scratches	240	60	300
Total	**1,440**	**360**	**1,800**

The classes are perfectly balanced, so plain accuracy is a fair headline metric and no class weighting or resampling was needed.

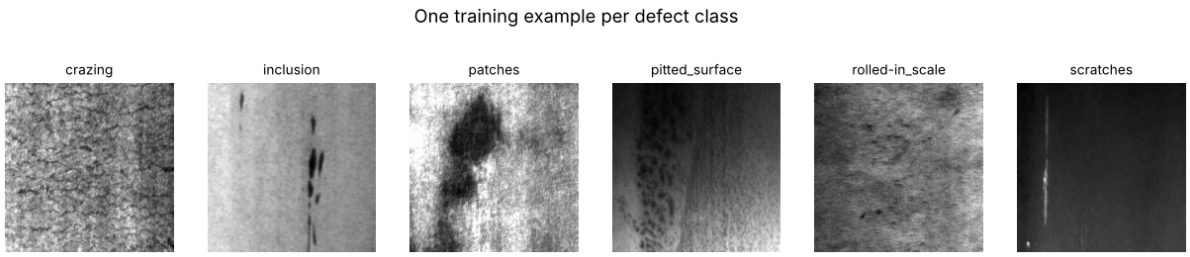


Figure 1. One training example per defect class.

4. Methodology

4.1 Data preparation

- **Loading:** torchvision ImageFolder, which assigns labels from folder names (crazing = 0 ... scratches = 5).
- **Format:** grayscale replicated to 3 channels, resized to 224 x 224 (MobileNetV2's pre-training resolution), normalised with ImageNet mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225].
- **Augmentation (training only):** random horizontal and vertical flips. Steel textures have no natural orientation, so flips never change the label.
- **Batching:** batch size 32. Training batches are shuffled so each batch mixes classes (ImageFolder otherwise returns images grouped by class). Validation is not shuffled, which keeps evaluation reproducible and lets predictions be traced back to files.

4.2 Model selection: MobileNetV2

MobileNetV2 (Sandler et al., 2018) with ImageNet-1k weights was used, as suggested. It suits this task for three reasons:

- It is lightweight (3.5M parameters), trains comfortably on a CPU, and is realistic for edge inspection hardware.
- Its inverted residual blocks with linear bottlenecks preserve fine detail through narrow layers, which helps with texture cues such as cracks, scratches and pits.
- ImageNet's low- and mid-level filters (edges, gradients, repeating patterns) transfer well to industrial textures.

4.3 Freezing strategy and classifier head

Partial transfer learning was applied. The whole convolutional feature extractor was frozen and only a new head was trained.

- **Frozen backbone:** all parameters in model.features have requires_grad = False. The backbone is also kept in eval() mode during training so its BatchNorm running statistics stay at their ImageNet values. Without this step, BatchNorm would keep adapting to small batches even though its weights are frozen.
- **New head:** Dropout(0.3) → Linear(1280 → 256) → ReLU → Dropout(0.3) → Linear(256 → 6). The small hidden layer lets the model combine ImageNet features non-linearly for steel textures, and dropout regularises it.

Block	Trainable parameters	Frozen parameters
features (MobileNetV2 backbone)	0	2,223,872
classifier (new head)	329,478	0
Total	**329,478 (12.9%)**	**2,223,872 (87.1%)**

4.4 Training configuration

Setting	Value
Loss	Cross-entropy (multi-class)
Optimizer	Adam, learning rate 1e-3, weight decay 1e-4 (head parameters only)
Epochs / batch size	12 / 32
Device	CPU (GPU used automatically if available)
Checkpointing	Weights with the best validation accuracy kept; final-epoch weights also kept for comparison
Reproducibility	Fixed random seed 42; two independent runs gave identical results

5. Results

5.1 Training behaviour

Training converged very quickly. The loss fell from 0.391 in epoch 1 to 0.037 in epoch 2 and settled near 0.005. Validation accuracy was at least 97.2% in every epoch and peaked at 100% in epoch 11, which also had the lowest validation loss (0.0082).

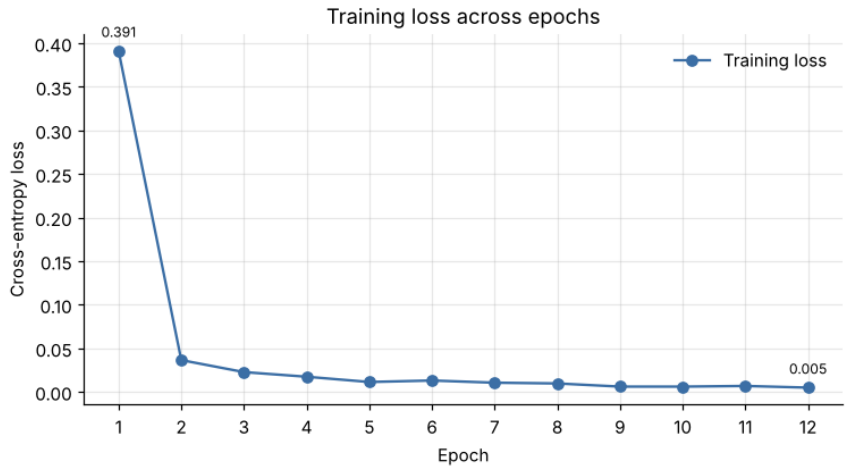


Figure 2. Training loss across epochs.

Epoch	Train loss	Train accuracy	Validation loss	Validation accuracy
1	0.3913	90.35%	0.0483	99.72%
2	0.0365	99.31%	0.0486	98.06%
3	0.0227	99.38%	0.0269	99.72%
4	0.0174	99.65%	0.0641	97.22%
5	0.0114	99.86%	0.0264	98.89%
6	0.0131	99.51%	0.0157	99.72%
7	0.0106	99.65%	0.0346	98.06%

Epoch	Train loss	Train accuracy	Validation loss	Validation accuracy
8	0.0097	99.79%	0.0325	98.33%
9	0.0061	99.86%	0.0579	97.78%
10	0.0061	99.86%	0.0118	99.44%
11	0.0068	99.79%	0.0082	100.00%
12	0.0049	99.93%	0.0130	99.72%

Validation loss is below training loss in epoch 1 because training loss is averaged while the head is still learning from random initialisation, and dropout and augmentation are active only during training.

5.2 Evaluation metrics

Metric	Best checkpoint (epoch 11)	Final epoch (12)
Accuracy	100.00% (360 / 360)	99.72% (359 / 360)
Macro F1-score	1.000	0.997
Validation loss	0.0082	0.0130
Per-class precision and recall	1.00 for every class	One inclusion image predicted as scratches

Caveat: the best checkpoint was chosen using validation accuracy, so its 100% score is slightly optimistic. The final-epoch model involves no selection and scores 99.72%. A fair estimate of expected performance on new images from the same source is therefore about 98-100%.

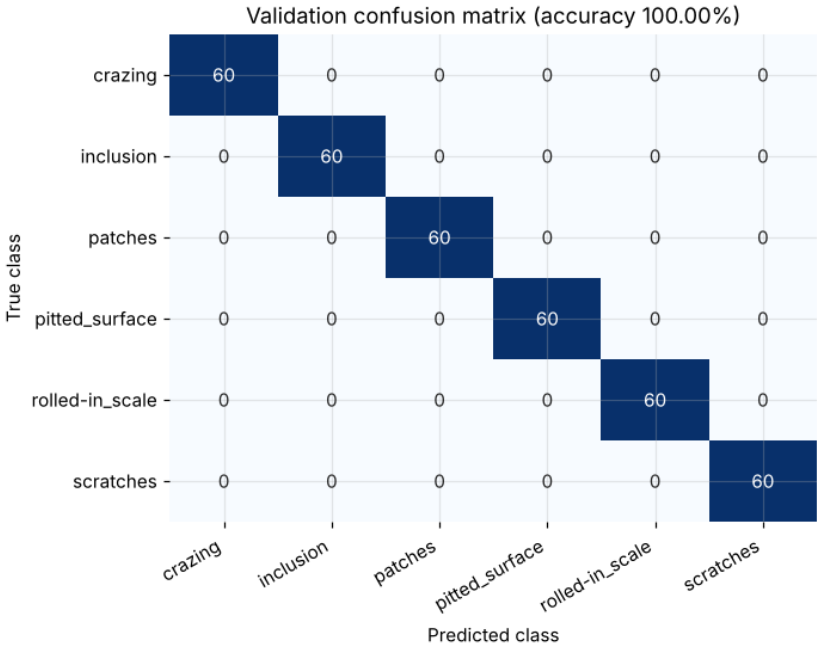


Figure 3. Validation confusion matrix for the best checkpoint.

5.3 Bias-variance diagnostics

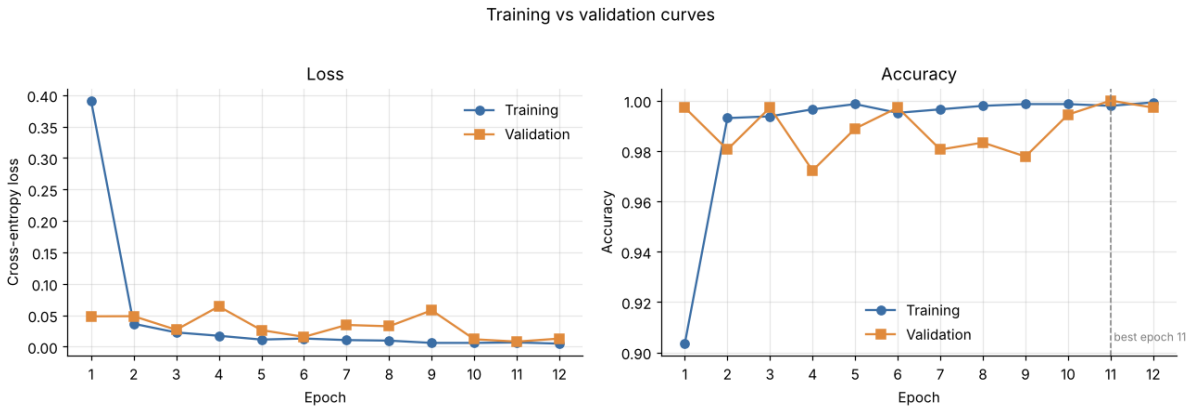


Figure 4. Training and validation loss (left) and accuracy (right).

Check	Evidence	Interpretation
Bias (underfitting?)	Training accuracy 99.9%, training loss 0.005	Low bias: the model has enough capacity
Variance (overfitting?)	Final gap +0.21 pp; gap never above 2.4 pp; validation loss 0.008-0.064, no upward trend	Low variance: the model generalises
Validation swings	Accuracy moves between 97.2% and 100%	1 image = 0.28 pp, so swings are 3-7 images: sampling noise, not a trend
Class balance	240 / 60 images in every class	Accuracy is not inflated by a majority class

Overall the model sits in the good-fit region. Freezing the 2.2M-parameter backbone is the main reason variance stays low with only 1,440 training images.

5.4 Prediction analysis

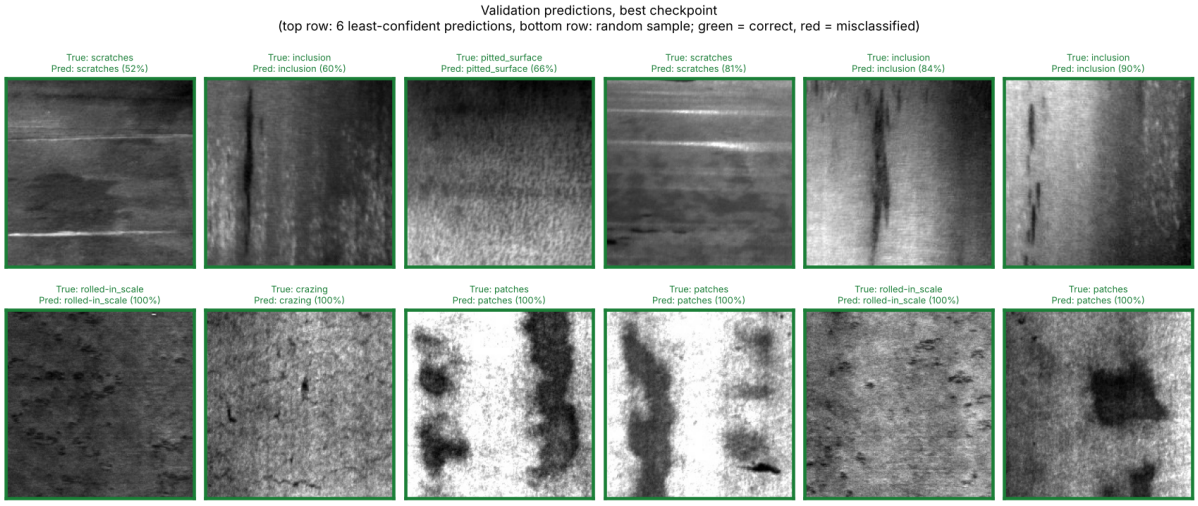


Figure 5. Validation predictions. Top row: the six least-confident predictions; bottom row: a random sample. Green borders mark correct predictions.

- All six least-confident predictions (52% to 90% confidence) are still correct. Mean confidence is 99.3%, and only 5 of 360 images fall below 90%.

- Uncertainty concentrates on inclusion vs scratches: both are thin, elongated streaks along the rolling direction (dark for inclusions, bright for scratches). One pitted-surface image is also less certain.
- The single error made by the final-epoch model is inclusion_283.jpg, predicted as scratches with 77% confidence. It shows one long, dark vertical line, a genuinely borderline case.
- Patches, crazing and rolled-in scale are recognised with essentially 100% confidence because their textures are distinctive.

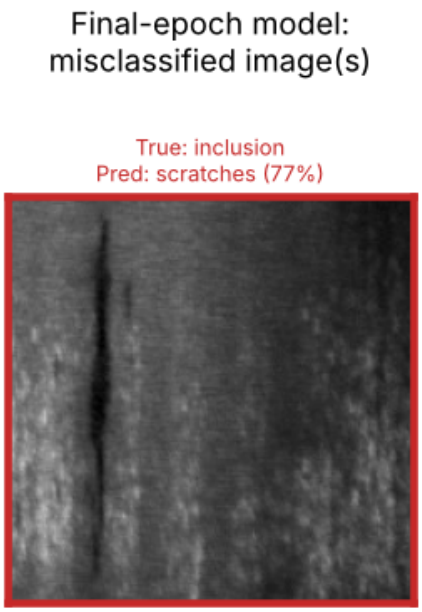


Figure 6. The only validation image misclassified by the final-epoch model.

6. Conclusions and insights

- **Transfer learning is highly effective here.** Features learned from natural images give near-perfect steel defect classification with only 240 images per class and no backbone fine-tuning.
- **Partial transfer learning is the right trade-off.** Training only 12.9% of the parameters kept training fast on a CPU and kept variance low on a small dataset.
- **The hardest distinction is inclusion vs scratches.** These two classes account for nearly all low-confidence predictions and for the only final-epoch error.
- **Model confidence is meaningful.** Low confidence coincides with genuinely ambiguous images, so a confidence threshold can route about 1-2% of parts to human inspectors and automate the rest.
- **Business value.** A small, fast and consistent model that can run on edge hardware at the line, reducing inspection cost and variability between inspectors.

7. Assumptions

- PyTorch and torchvision were used (the assignment allows PyTorch or TensorFlow).

- The dataset sits in a folder named NEU-DET next to the notebook, with train/images/<class> and validation/images/<class> sub-folders; the loader also accepts the layout without the images level.
- The provided split (240 / 60 per class) was used as is. There is no separate test set, so validation data serves as the held-out evaluation set and is never used to update weights.
- Pre-trained weights: the notebook first requests torchvision's official IMAGENET1K_V1 MobileNetV2 weights. Where that download is blocked (as in the environment used to execute the notebook), it loads a local copy of ImageNet-pretrained MobileNetV2 weights (same architecture, 72.2% ImageNet top-1, from the public d-li14/mobilenetv2.pytorch release) mapped one-to-one onto torchvision's layers.
- Only flip augmentation was applied, and only to training images.
- Results are seeded for reproducibility, but exact numbers may differ slightly on other hardware or library versions.

8. Limitations and next steps

- The 100% result is mildly optimistic because validation data also selected the checkpoint. A separate test set or k-fold cross-validation would give an unbiased estimate.
- All images come from one dataset and capture setup. Deployment would need testing under different lighting, cameras and steel grades.
- If accuracy dropped on new data: unfreeze the last few inverted-residual blocks with a small learning rate, add brightness and contrast augmentation, and collect more inclusion and scratch examples.
- The model assigns one label per image and does not locate defects; detection or segmentation models would be needed for that.

References

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR. arXiv:1801.04381.

Song, K. and Yan, Y. (2013). A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. Applied Surface Science, 285, 858-864. (NEU Surface Defect Database.)

Kingma, D. P. and Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980.

PyTorch and torchvision documentation: torchvision.models.mobilenet_v2, torchvision.transforms, torch.utils.data.DataLoader.

Github link: https://github.com/bulah16/C7-Project